Supplementary Material for "TWAPPA-FRL: Trust-Weighted Privacy-Preserving Aggregation for Federated Reinforcement Learning in Collaborative Intrusion Detection"

Piyumi M. S. Rajapaksha Artur Iasenovets Yinxue Yi Fei Tang

1 Detailed Algorithms

This section provides the workflow algorithms for the $ep_{\text{twa-ppa}}$ execution path.

1.1 TWA Component Definitions

Let

$$\text{clip}_{[0,1]}(x) = \min(1, \max(0, x)).$$

The fixed public protocol constant $\delta_0 = 10^{-8}$ is the numerical stabilizer. The parameter domains are $\alpha_k \geq 0$ with $\sum_k \alpha_k = 1$, $\beta, \lambda \in [0,1]$, $\rho \geq 0$, $\tau > 0$, $r_{\max} > r_{\min}$, $w_{\max} > 0$, and $C > 0$.

For per-step rewards in $[-2, 3]$ and at most $t_{\max}$ steps per episode, $r_{\min} = -2t_{\max}$ and $r_{\max} = 3t_{\max}$. For the reported five-client configuration $t_{\max} = 16$, hence $(r_{\min}, r_{\max}) = (-32, 48)$.

For the latest reward-history window $\mathcal{H}_{\text{rew},i}^{(r)} = (\bar{r}_i^{(t)})_{t=\max(0,r-4)}^{r}$, reward stability is

$$s_i^{\text{raw}} = \begin{cases} 0.7, & |\mathcal{H}_{\text{rew},i}^{(r)}| < 2, \\ \text{clip}_{[0,1]}\left[\dfrac{1}{1 + \text{std}(\mathcal{H}_{\text{rew},i}^{(r)})/(\text{mean}\,|\mathcal{H}_{\text{rew},i}^{(r)}| + \delta_0)}\right], & \text{otherwise,} \end{cases} \qquad S_i = s_i^{\text{raw}}.$$

For a window $\mathcal{H}$ of length n, the calculation uses the population standard deviation

$$\text{std}(\mathcal{H}) = \left[n^{-1} \sum_{x \in \mathcal{H}} (x - \text{mean}(\mathcal{H}))^2\right]^{1/2}.$$

The quality component is $Q_i = \text{clip}_{[0,1]}((\bar{r}_i - r_{\min})/(r_{\max} - r_{\min}))$. For the clipped update $\mathbf{z}_i$ and the reference update $\mathbf{v} = \mathsf{flatten}(\Delta\theta_{\text{agg}}^{(r-1)})$,

$$M_i = \text{clip}_{[0,1]}\left(\frac{1}{2}\left[1 + \frac{\mathbf{z}_i^\top \mathbf{v}}{\|\mathbf{z}_i\|_2 \cdot \|\mathbf{v}\|_2 + \delta_0}\right]\right), \qquad H_i = \begin{cases} 0.5, & b_i^{(r)} = 1, \\ T_i^{(r-1)}, & b_i^{(r)} = 0, \end{cases}.$$

Let $\mathcal{I}_{r-1}$ be the set of client indices for which $\mathcal{TC}$ stores a prior trust value. Then $b_i^{(r)} = 1$ exactly when $i \notin \mathcal{I}_{r-1}$, and $b_i^{(r)} = 0$ otherwise. If local validation data $\mathcal{V}_i$ are available, $\mathsf{LocalF1Metric}(\mathcal{V}_i)$ is the macro-F1 score

$$F_i^{\text{raw}} = \frac{1}{J} \sum_{c=1}^{J} \frac{2p_{i,c}q_{i,c}}{p_{i,c} + q_{i,c}},$$

where $p_{i,c}$ and $q_{i,c}$ are class-c precision and recall; a term is zero when its denominator is zero. Otherwise, $F_i^{\text{raw}} = 0.5$, and $F_i = \text{clip}_{[0,1]}(F_i^{\text{raw}})$. Finally,

$$d_i = \frac{|g_i - g_{\text{med}}|}{g_{\text{med}} + \delta_0}, \qquad G_i = \frac{1}{1 + d_i}.$$

These local transforms define the distributed protocol.

1.2 Protocol Algorithms

For compact workflow notation, let

$$\mathcal{X}_i^{(r)} = \left(\Delta\theta_i^{(r)}, |\mathcal{D}_i|, \bar{r}_i, \mathcal{H}_{\text{rew},i}^{(r)}, F_i^{\text{raw}}, g_i, g_{\text{med}}, \Delta\theta_{\text{agg}}^{(r-1)}, T_i^{(r-1)}, b_i^{(r)}\right).$$

The following algorithms use these definitions when invoking TrustScore. Before round 0, initialize the public aggregate state $\Delta\theta_{\text{agg}}^{(-1)} = \mathbf{0}$, the stored-client index set $\mathcal{I}_{-1} = \varnothing$, and each client-local reward history $\mathcal{H}_{\text{rew},i}^{(-1)} = [\,]$.

Algorithm S1 TWA-PPA Workflow — $ep_{\text{twa-ppa}}$ main path

Require: $\theta^{(r)}$; $\Delta\theta_{\text{agg}}^{(r-1)}$; pk; pp; sk
Require: $\mathcal{I}_{r-1}$; $\{T_i^{(r-1)}\}_{i\in\mathcal{I}_{r-1}}$; $\{\alpha_k, \beta, \lambda, \rho, \tau, r_{\min}, r_{\max}, w_{\max}, C\}$;
Require: $\{(\mathcal{DC}_i, \mathcal{E}_i, \mathcal{D}_i, \mathcal{V}_i)\}_{i=1}^K$
Ensure: $\theta^{(r+1)}$

Broadcast $\theta^{(r)}$ and the public $\Delta\theta_{\text{agg}}^{(r-1)}$ to all $\mathcal{DC}_i$
for each client $i \in \{1, \ldots, K\}$ **in parallel do**
  $(\Delta\theta_i^{(r)}, \bar{r}_i, \bar{\ell}_i) \leftarrow \textsc{LocalTrain}(\theta^{(r)}, E, \mathcal{E}_i, \mathcal{D}_i)$
  $\mathcal{H}_{\text{rew},i}^{(r)} \leftarrow \mathsf{tail}_5(\mathcal{H}_{\text{rew},i}^{(r-1)} \,\|\, [\bar{r}_i])$ locally
  $F_i^{\text{raw}} \leftarrow \mathsf{LocalF1Metric}(\mathcal{V}_i)$ if available; otherwise $F_i^{\text{raw}} \leftarrow 0.5$
  $g_i \leftarrow \|\mathsf{flatten}(\Delta\theta_i^{(r)})\|_2$
end for
$\mathcal{TC}$ receives $\{g_i\}_{i=1}^K$
$\mathcal{TC}$ sets $b_i^{(r)} \leftarrow 1$ if $i \notin \mathcal{I}_{r-1}$, and $b_i^{(r)} \leftarrow 0$ otherwise
$\mathcal{TC}$ sets $T_i^{(r-1)} \leftarrow 0.5$ when $b_i^{(r)} = 1$
$g_{\text{med}} \leftarrow \mathsf{median}(\{g_i\}_{i=1}^K)$
$\mathcal{TC}$ distributes g_{med} to all $\mathcal{DC}_i$
$\mathcal{TC}$ provides $(T_i^{(r-1)}, b_i^{(r)})$ to each $\mathcal{DC}_i$
for each client $i \in \{1, \ldots, K\}$ **in parallel do**
  $(\bar{\Delta\theta}_i^{(r)}, \hat{w}_i^{(r)}, T_i^{(r)}) \leftarrow \textsc{TrustScore}(\mathcal{X}_i^{(r)}, \Phi_{\text{TWA}})$
end for
Each $\mathcal{DC}_i$ sends $T_i^{(r)}$ directly to $\mathcal{TC}$
Each $\mathcal{DC}_i$ sends $(\hat{w}_i^{(r)}, |\mathcal{D}_i|)$ to $\mathcal{AC}$
$\mathcal{TC}$ stores $\{T_i^{(r)}\}_{i=1}^K$ for use as the previous trust state in round $r+1$
$\{w_i^{(r)}\}_{i=1}^K \leftarrow \textsc{TrustWeightNorm}(\{\hat{w}_i^{(r)}\}_{i=1}^K, \{|\mathcal{D}_i|\}_{i=1}^K, w_{\max})$
for each client $i \in \{1, \ldots, K\}$ **in parallel do**
  $\{\mathsf{ct}_{i,\ell}^{(r)}\}_{\ell=0}^{L-1} \leftarrow \text{CKKSEncryptDelta}(\bar{\Delta\theta}_i^{(r)}, pk, P, m_{\text{slot}})$
end for
Each $\mathcal{DC}_i$ sends $\{\mathsf{ct}_{i,\ell}^{(r)}\}_{\ell=0}^{L-1}$ to $\mathcal{AC}$
$\{\mathsf{ct}_{\text{agg},\ell}^{(r)}\}_{\ell=0}^{L-1} \leftarrow \text{HEWeightedAggregate}(\{\mathsf{ct}_{i,\ell}^{(r)}\}, \{w_i^{(r)}\}, \text{pp})$
$\mathcal{AC}$ sends only $\{\mathsf{ct}_{\text{agg},\ell}^{(r)}\}_{\ell=0}^{L-1}$ to $\mathcal{KA}$
$\Delta\theta_{\text{HE}}^{(r)} \leftarrow \textsc{DecryptAggregate}(\{\mathsf{ct}_{\text{agg},\ell}^{(r)}\}_{\ell=0}^{L-1}, sk, P, m_{\text{slot}})$
$\mathcal{KA}$ returns $\Delta\theta_{\text{HE}}^{(r)}$ to $\mathcal{AC}$
$\Delta\theta_{\text{agg}}^{(r)} \leftarrow \Delta\theta_{\text{HE}}^{(r)}$
$\theta^{(r+1)} \leftarrow \theta^{(r)} + \Delta\theta_{\text{agg}}^{(r)}$
Broadcast $\theta^{(r+1)}$ to all $\mathcal{DC}_i$
Log accuracy, macro-F1, reward
return $\theta^{(r+1)}$

Algorithm S2 LocalTrain at $\mathcal{DC}_i$

Require: $\theta^{(r)}$; E; $\mathcal{E}_i$; $\mathcal{D}_i$
Ensure: $\Delta\theta_i^{(r)}$; $\bar{r}_i$; $\bar{\ell}_i$

$\theta_i \leftarrow \theta^{(r)}$; $\mathcal{R}_{\text{ep}} \leftarrow [\,]$; $\mathcal{L}_{\text{train}} \leftarrow [\,]$
for $e = 1, \ldots, E$ **do**
  $s \leftarrow \mathcal{E}_i.\mathsf{reset}()$; $d \leftarrow \mathsf{false}$; $r_e \leftarrow 0$; $t \leftarrow 0$
  while $\neg d \wedge t < t_{\max}$ **do**
    $a \leftarrow \pi_i^{\varepsilon\text{-greedy}}(s; \theta_i)$
    $(s', r, d) \leftarrow \mathcal{E}_i.\mathsf{step}(a)$
    $\mathcal{D}_i.\mathsf{store}(s, a, r, s', d)$
    $\ell_{\text{loss}} \leftarrow \mathsf{DQNTrainStep}(\theta_i, \mathcal{D}_i)$
    if $\ell_{\text{loss}} \neq \varnothing$ **then**
      $\mathcal{L}_{\text{train}}.\mathsf{append}(\ell_{\text{loss}})$
    end if
    $r_e \leftarrow r_e + r$; $s \leftarrow s'$; $t \leftarrow t + 1$
  end while
  $\mathcal{R}_{\text{ep}}.\mathsf{append}(r_e)$
end for
$\Delta\theta_i^{(r)} \leftarrow \theta_i^{\text{local}} - \theta^{(r)}$
$\bar{r}_i \leftarrow \mathsf{mean}(\mathcal{R}_{\text{ep}})$; $\bar{\ell}_i \leftarrow \mathsf{mean}(\mathcal{L}_{\text{train}})$
return $(\Delta\theta_i^{(r)}, \bar{r}_i, \bar{\ell}_i)$

Algorithm S3 TrustScore for client i

Require: $\Delta\theta_i^{(r)}$; $|\mathcal{D}_i|$; $\bar{r}_i$; $\mathcal{H}_{\text{rew},i}^{(r)}$; F_i^{raw}; g_i; g_{med}; $\Delta\theta_{\text{agg}}^{(r-1)}$; $T_i^{(r-1)}$; $b_i^{(r)}$; $\{\alpha_k, \beta, \lambda, \rho, \tau, r_{\min}, r_{\max}, w_{\max}, C\}$
Ensure: $\bar{\Delta\theta}_i^{(r)}$; $\hat{w}_i$; T_i

$\mathbf{z}_i \leftarrow \mathsf{flatten}(\Delta\theta_i^{(r)})$
if $\|\mathbf{z}_i\|_2 > 0$ **then**
  $\mathbf{z}_i \leftarrow \mathbf{z}_i \cdot \min(1, C/\|\mathbf{z}_i\|_2)$
else
  $\mathbf{z}_i \leftarrow \mathbf{0}$
end if
Compute s_i^{raw} from $\mathcal{H}_{\text{rew},i}^{(r)}$; $S_i \leftarrow s_i^{\text{raw}}$
$Q_i \leftarrow \text{clip}_{[0,1]}((\bar{r}_i - r_{\min})/(r_{\max} - r_{\min}))$
$\mathbf{v} \leftarrow \mathsf{flatten}(\Delta\theta_{\text{agg}}^{(r-1)})$; $M_i \leftarrow \text{clip}_{[0,1]}\big(\frac{1}{2}[1 + \mathbf{z}_i^\top \mathbf{v}/(\|\mathbf{z}_i\|_2 \cdot \|\mathbf{v}\|_2 + \delta_0)]\big)$
$H_i \leftarrow 0.5$ if $b_i^{(r)} = 1$; otherwise $H_i \leftarrow T_i^{(r-1)}$
$F_i \leftarrow \text{clip}_{[0,1]}(F_i^{\text{raw}})$
$d_i \leftarrow |g_i - g_{\text{med}}|/(g_{\text{med}} + \delta_0)$; $G_i \leftarrow 1/(1 + d_i)$
$T_{i,\text{raw}} \leftarrow \alpha_S S_i + \alpha_Q Q_i + \alpha_M M_i + \alpha_H H_i + \alpha_F F_i + \alpha_G G_i$
$T_i \leftarrow T_{i,\text{raw}}$ if $b_i^{(r)} = 1$; otherwise $T_i \leftarrow \beta T_i^{(r-1)} + (1 - \beta) T_{i,\text{raw}}$
$\bar{\Delta\theta}_i^{(r)} \leftarrow \mathsf{unflatten}(\mathbf{z}_i)$
if $T_i < \lambda$ **then**
  $\hat{w}_i \leftarrow 0$
else
  $\hat{w}_i \leftarrow |\mathcal{D}_i| T_i^{\rho/\tau}$
end if
return $(\bar{\Delta\theta}_i^{(r)}, \hat{w}_i, T_i)$

Algorithm S4 TrustWeightNorm

Require: $\{\hat{w}_i\}_{i=1}^{K}$; $\{|\mathcal{D}_i|\}_{i=1}^{K}$; $w_{\max} > 0$
Ensure: $\{w_i\}_{i=1}^{K}$

$W \leftarrow \sum_{i=1}^{K} \hat{w}_i$; $S_D \leftarrow \sum_{i=1}^{K} |\mathcal{D}_i|$
require $S_D > 0$
if $W > 0$ **then**
  $a_i \leftarrow \hat{w}_i / W$ for all i; $\mathcal{A} \leftarrow \{i : \hat{w}_i > 0\}$
else
  $a_i \leftarrow |\mathcal{D}_i| / S_D$ for all i; $\mathcal{A} \leftarrow \{i : |\mathcal{D}_i| > 0\}$
end if
require $w_{\max} \geq 1/|\mathcal{A}|$; $w_i \leftarrow 0$ for all i; $R \leftarrow 1$
while $\mathcal{A} \neq \varnothing$ **do**
  $p_i \leftarrow R a_i / \sum_{j \in \mathcal{A}} a_j$ for $i \in \mathcal{A}$; $\mathcal{C} \leftarrow \{i \in \mathcal{A} : p_i > w_{\max}\}$
  if $\mathcal{C} = \varnothing$ **then**
    $w_i \leftarrow p_i$ for $i \in \mathcal{A}$; $\mathcal{A} \leftarrow \varnothing$
  else
    $w_i \leftarrow w_{\max}$ for $i \in \mathcal{C}$; $R \leftarrow R - |\mathcal{C}| w_{\max}$
    $\mathcal{A} \leftarrow \mathcal{A} \setminus \mathcal{C}$
  end if
end while
return $\{w_i\}_{i=1}^{K}$

Algorithm S5 CKKSEncryptDelta at $\mathcal{DC}_i$

Require: $\bar{\Delta}\theta_i$; pk; $m_{\text{slot}} = 4096$; $P = 53{,}636$
Ensure: $\{\mathsf{ct}_{i,\ell}\}_{\ell=0}^{L-1}$; $L = \lceil P / m_{\text{slot}} \rceil = 14$

$\mathbf{z}_i \leftarrow \mathsf{flatten}(\bar{\Delta}\theta_i)$
$L \leftarrow \lceil P / m_{\text{slot}} \rceil$
for $\ell = 0, \ldots, L-1$ **do**
  $\mathbf{z}_{i,\ell} \leftarrow \mathbf{z}_i[\ell m_{\text{slot}} : (\ell+1) m_{\text{slot}}]$
  if $\ell = L - 1$ **then**
    zero-pad $\mathbf{z}_{i,\ell}$ to length m_{slot}
  end if
  $\mathsf{ct}_{i,\ell} \leftarrow \mathrm{Enc}_{pk}(\mathbf{z}_{i,\ell})$
end for
return $\{\mathsf{ct}_{i,\ell}\}_{\ell=0}^{L-1}$

Algorithm S6 HEWeightedAggregate at $\mathcal{AC}$

Require: $\{\{\mathsf{ct}_{i,\ell}\}_{\ell=0}^{L-1}\}_{i=1}^{K}$; $\{w_i\}_{i=1}^{K}$; pp
Ensure: $\{\mathsf{ct}_{\text{agg},\ell}\}_{\ell=0}^{L-1}$

for $\ell = 0, \ldots, L-1$ **do**
  $\mathsf{ct}_{\text{agg},\ell} \leftarrow w_1 \otimes \mathsf{ct}_{1,\ell}$
  for $i = 2, \ldots, K$ **do**
    $\mathsf{ct}_{\text{agg},\ell} \leftarrow \mathsf{ct}_{\text{agg},\ell} \oplus (w_i \otimes \mathsf{ct}_{i,\ell})$
  end for
end for
return $\{\mathsf{ct}_{\text{agg},\ell}\}_{\ell=0}^{L-1}$

Algorithm S7 DecryptAggregate at $\mathcal{KA}$

Require: $\{\mathsf{ct}_{\mathrm{agg},\ell}\}_{\ell=0}^{L-1}$; sk; P; m_{slot}
Ensure: $\Delta\theta_{\mathrm{HE}}$
1: **for** $\ell = 0, \ldots, L-1$ **do**
2: $\quad \hat{\mathbf{z}}_\ell \leftarrow \mathrm{Dec}_{sk}(\mathsf{ct}_{\mathrm{agg},\ell})$
3: **end for**
4: $\mathbf{z}_{\mathrm{agg}} \leftarrow \mathsf{concat}(\hat{\mathbf{z}}_0, \ldots, \hat{\mathbf{z}}_{L-1})[:P]$
5: $\Delta\theta_{\mathrm{HE}} \leftarrow \mathsf{reconstruct}(\mathbf{z}_{\mathrm{agg}})$
6: **return** $\Delta\theta_{\mathrm{HE}}$

Path-specific execution location. Both TWA paths use the same six-component transforms above, the same trust combination and smoothing rule in Algorithm S3, and the same preliminary and final weight calculations. In ep_{twa}, $\mathcal{TC}$ evaluates these calculations on the plaintext norm-bounded updates and associated diagnostics. In $ep_{\text{twa-ppa}}$, each $\mathcal{DC}_i$ evaluates Algorithm S3 locally before encrypting its norm-bounded update; the previous public aggregate supplies the alignment reference. Thus, the paths differ in execution location and protected transport, not in the six-component TWA6 scoring formula. The shared evaluation routine retains behavior-consistency and reward-consistency values only as diagnostics: both corresponding coefficients are zero, and the logged consistency factor is identically one.

2 Protocol Metadata and Execution Paths

2.1 Information Classes

The protocol distinguishes the following information classes:

TWA policy. The configuration

$$\Phi_{\mathrm{TWA}} = \{\alpha_k, \beta, \lambda, \rho, \tau, r_{\min}, r_{\max}, w_{\max}, C\}$$

contains the public TWA policy parameters, where $\{\alpha_k\}$ define the trust components, β controls temporal smoothing, λ is the gating threshold, ρ controls trust-weight sharpening, τ is the power-temperature parameter fixed to 1 in the reported experiments, $r_{\min}$ and $r_{\max}$ are public reward-bound constants, $w_{\max}$ limits individual influence, and C is the update-norm bound. These public TWA policy constants are maintained by $\mathcal{TC}$ and distributed to the entities executing the corresponding TWA routines.

Cross-round trust state. The state

$$\mathcal{S}_{\mathrm{TWA}}^{(r)} = \{T_i^{(r)}\}_{i \in \mathcal{I}_r}, \qquad \mathcal{I}_r \subseteq \{1, \ldots, K\},$$

is a partial mapping of stored client indices to their current trust scores. In $ep_{\text{twa-ppa}}$, each client sends $T_i^{(r)}$ directly to $\mathcal{TC}$, which stores it for that participating client. After a full fixed-client round, $\mathcal{I}_r = \{1, \ldots, K\}$. Round $r+1$ uses $\mathcal{S}_{\mathrm{TWA}}^{(r)} = \{T_i^{(r)}\}_{i \in \mathcal{I}_r}$ as its previous trust state. In each round, $\mathcal{TC}$ derives $b_i^{(r)} = \mathbf{1}[i \notin \mathcal{I}_{r-1}]$ and supplies it alongside $T_i^{(r-1)}$.

Generated trust metadata. Local execution of TrustScore produces

$$\mathcal{M}_i^{(r)} = \left(T_i^{(r)}, \hat{w}_i^{(r)}\right),$$

where $T_i^{(r)}$ is the current trust score and $\hat{w}_i^{(r)}$ is the preliminary aggregation weight.

Update-norm diagnostics. The update-norm diagnostics are

$$\mathcal{G}^{(r)} = \left(\{g_i^{(r)}\}_{i=1}^K, g_{\mathrm{med}}^{(r)}\right), \qquad g_i^{(r)} = \left\|\mathsf{flatten}(\Delta\theta_i^{(r)})\right\|_2.$$

In $ep_{\text{twa-ppa}}$, each client releases $g_i^{(r)}$ to $\mathcal{TC}$, which computes and distributes

$$g_{\mathrm{med}}^{(r)} = \mathsf{median}\left(\{g_i^{(r)}\}_{i=1}^K\right).$$

The remaining TrustScore inputs, including $\bar{r}_i$, $\bar{\ell}_i$, and F_i^{raw}, remain local.

Destination-specific client release. In $ep_{\text{twa-ppa}}$, the client's trust-related data are released according to their destination:

$$\mathcal{R}^{(r)}_{i\to\mathcal{TC}} = \left(g_i^{(r)}, T_i^{(r)}\right), \qquad \mathcal{R}^{(r)}_{i\to\mathcal{AC}} = \left(\hat{w}_i^{(r)}, |\mathcal{D}_i|\right).$$

$\mathcal{TC}$ uses $g_i^{(r)}$ to compute $g_{\text{med}}^{(r)}$ and stores $\{T_i^{(r)}\}_{i=1}^K$ as cross-round state for use in the next round. $\mathcal{AC}$ uses $\{(\hat{w}_i^{(r)}, |\mathcal{D}_i|)\}_{i=1}^K$ to derive the final aggregation weights $\{w_i^{(r)}\}_{i=1}^K$. The term *client trust metadata*, used in the notation table and information classes, denotes the union of these destination-specific releases, rather than a single tuple observed by every protocol entity.

Aggregation weights. The final weights are

$$\mathcal{W}^{(r)} = \{w_i^{(r)}\}_{i=1}^K.$$

In $ep_{\text{twa-ppa}}$, they are derived by $\mathcal{AC}$ from $\{\hat{w}_i^{(r)}\}$ and $\{|\mathcal{D}_i|\}$ through TrustWeightNorm. In ep_{twa}, $\mathcal{TC}$ derives them through the centralized trust-scoring variant followed by TrustWeightNorm. They are distinct from the client-generated preliminary weights.

Model-update payloads. The path-dependent update payloads comprise:

- the raw client update $\Delta\theta_i^{(r)}$;
- the norm-bounded update $\bar{\Delta\theta}_i^{(r)}$;
- the encrypted client chunks $\{\mathsf{ct}_{i,\ell}^{(r)}\}_{\ell=0}^{L-1}$;
- the aggregate ciphertext chunks $\{\mathsf{ct}_{\text{agg},\ell}^{(r)}\}_{\ell=0}^{L-1}$; and
- the recovered aggregate $\Delta\theta_{\text{HE}}^{(r)} = \Delta\theta_{\text{agg}}^{(r)}$.

Public round state. The public round state includes the global model $\theta^{(r)}$ and the previous aggregate $\Delta\theta_{\text{agg}}^{(r-1)}$, which is used as the update-similarity reference in TrustScore.

CKKS key material. The CKKS material comprises the public context pp, public key pk, and secret key sk:

$$\mathcal{K}_{\text{CKKS}} = \{pp, pk, sk\}.$$

The public context and pk are available to the clients and $\mathcal{AC}$, while sk is retained only by $\mathcal{KA}$.

2.2 Security Scope Across Rounds

Single-round scope. For an honest-but-curious $\mathcal{AC}$, the encrypted-path round view comprises the client ciphertext chunks $\{\mathsf{ct}_{i,\ell}^{(r)}\}_{i,\ell}$, aggregate ciphertext chunks $\{\mathsf{ct}_{\text{agg},\ell}^{(r)}\}_{\ell}$, its destination-specific metadata $\{(\hat{w}_i^{(r)}, |\mathcal{D}_i|)\}_i$, final weights $\{w_i^{(r)}\}_i$, and public context and round state. Under the CKKS IND-CPA/RLWE assumption [1, 2] and the key boundary in which $\mathcal{AC}$ does not hold sk, this view does not enable recovery of an individual plaintext update from the ciphertext observations. This is a ciphertext-confidentiality claim; it does not hide the designated metadata or the aggregate update.

Multi-round scope. Fresh CKKS encryption randomness is sampled for each client chunk in every round. Thus, a standard hybrid argument applies to the ciphertext portions of equal-length execution traces. It does not imply that all protocol observations are indistinguishable across rounds or that information cannot accumulate. In particular, the global models and recovered aggregates reveal $\Delta\theta_{\text{agg}}^{(r)}$, and the protocol releases update norms, trust values, preliminary weights, replay sizes, and final weights over time. Those values may reveal coarse training or reliability behavior and remain outside the ciphertext-confidentiality claim.

Leakage and excluded cases. In $ep_{\text{twa-ppa}}$, each client first sends $g_i^{(r)}$ to $\mathcal{TC}$, which returns the round median and prior trust state required for local scoring. The client then sends its resulting $T_i^{(r)}$ to $\mathcal{TC}$ and $(\hat{w}_i^{(r)}, |\mathcal{D}_i|)$ to $\mathcal{AC}$, as specified by Algorithm S1. The coordinator derives $\{w_i^{(r)}\}_i$, aggregates the ciphertexts, and forwards only aggregate ciphertext chunks to $\mathcal{KA}$; $\mathcal{KA}$ returns the recovered aggregate. Permissible leakage also includes ciphertext sizes and chunk counts, timing, participant count, selected execution path, and run metadata. The claim excludes $\mathcal{AC}$– $\mathcal{KA}$ collusion, delivery of individual client ciphertexts to $\mathcal{KA}$, release of sk to $\mathcal{AC}$, and collusion between $\mathcal{AC}$ and $K-1$ clients, which can reveal the remaining contribution from the aggregate.

2.3 Execution-Path Details

The baseline ep_0 aggregates raw client updates using uniform weights $w_i = 1/K$. The ep_{sdp} path applies uniform clipping and Gaussian perturbation before plaintext uniform aggregation, whereas ep_{twa} applies norm bounding and trust-derived aggregation weights in plaintext. The encrypted counterparts preserve the corresponding aggregation structure while replacing plaintext aggregation with CKKS-based PPA: ep_{ppa} securely aggregates raw updates with uniform weights, $ep_{\text{sdp-ppa}}$ securely aggregates statically perturbed updates, and $ep_{\text{twa-ppa}}$ securely aggregates norm-bounded updates using public trust-derived weights. The $ep_{\text{twa-ppa}}$ path evaluates the common TWA6 transforms locally through Algorithm S3, whereas ep_{twa} evaluates the same transforms at $\mathcal{TC}$, as specified in Section 1.1. Adversarial behavior is applied separately as an evaluation condition and does not define an additional execution path.

2.4 Adversarial Evaluation Details

The single-attack set is

$$\mathcal{A}_{\text{single}} = \{\text{SF}, \text{RNB}, \text{MR}, \text{LF}, \text{DaP}, \text{ReP}\},$$

denoting sign flipping, random-noise Byzantine updates, model replacement or scaled sign flipping, label flipping, data poisoning, and reward poisoning, respectively. The clean condition is denoted by $\varnothing$.

For sign flipping, the malicious client submits $-\Delta\theta_i^{(r)}$ after local delta computation and before TWA scoring or aggregation is applied.

For RNB-Primary, after all clients have produced their raw updates, the runner replaces the malicious vector by an isotropic Gaussian vector before TWA scoring or aggregation. If P is the update dimension and $g_{\text{med}}^{\text{ben}}$ is the median benign-update norm, each coordinate is sampled from $\mathcal{N}(0, \sigma^2)$ with

$$\sigma = 0.5\, g_{\text{med}}^{\text{ben}}/\sqrt{P}.$$

For model replacement, the runner submits $-8\Delta\theta_i^{(r)}$ at the same post-training, pre-scoring injection point. Thus, sign flipping, RNB, and model replacement alter the submitted update before TWA norm bounding and before plaintext or encrypted aggregation.

Label flipping and data poisoning are applied to the malicious client's fixed training partition before local environments and agents are instantiated. Label flipping maps every class to the next class in the fixed four-class ordering. Data poisoning adds independent Gaussian perturbations to every numeric training feature; for feature j, the noise standard deviation is $8(0.05)s_j = 0.4s_j$, where s_j is that feature's empirical standard deviation in the malicious partition. Neither attack modifies the shared test set. Reward poisoning is applied after local training but before trust scoring: the reported mean episode reward is replaced by $3t_i \times 8$, where t_i is the malicious client's realized local-step count. It therefore changes the reward-derived trust inputs without modifying the already computed model update.

For combined attacks, define

$$\mathcal{U} = \{\varnothing, \text{SF}, \text{RNB}, \text{MR}\}, \qquad \mathcal{L} = \{\text{LF}, \text{DaP}, \text{ReP}\},$$

where at most one update-level attack in $\mathcal{U}$ is active, while any subset of $\mathcal{L}$ may be composed. The resulting condition space is

$$\mathcal{A}_{\text{eval}} = \{(u, \ell) \mid u \in \mathcal{U},\ \ell \subseteq \mathcal{L}\},$$

containing 32 conditions including $(\varnothing, \varnothing)$.

This broader space formalizes how attacks could be composed, but no condition containing two or more attacks is used in the reported experiments. The robustness evaluation reports only the clean condition and the six standalone conditions in $\mathcal{A}_{\text{single}}$ under the same training schedule. Cumulative reward is the sum over rounds of the mean client episode reward. Section 2.5 distinguishes the operational low-trust gate from the reporting-only suppression cutoff.

2.5 Experimental Setup Details

Experiments 1–2 use the operational low-trust gate $\lambda = 10^{-8}$, below which a client receives zero preliminary aggregation weight. Separately, Experiment 2 uses 10^{-4} as a reporting-only cutoff for negligible combined malicious aggregation weight; it does not alter protocol execution.

The clean evaluation and six standalone-attack evaluations use the four-class CIC-IDS2017 subset [3, 4] comprising benign, DDoS, port-scan, and FTP brute-force traffic, represented by 78 numeric input features. The runner consumes five fixed, pre-generated non-i.i.d. training partitions

`client_0_train.csv` through `client_4_train.csv` and one shared test set. At load time, label strings are normalized, rows containing missing or non-finite values are discarded, and the numeric features are standardized before model input. The runner does not repartition examples at run time. Partition and test-set hashes are held fixed across execution paths, attack conditions, and seeds; within each seed, all paths and conditions also reuse the same saved initial model state. Clean runs treat all five clients as benign, whereas each adversarial run marks client 4 (zero-indexed) as malicious.

Each five-client Experiment 1 configuration is evaluated with seeds 42, 123, and 2025 for 30 federated rounds. A client performs one local ε-greedy episode per round, capped at 16 environment steps, followed by one auxiliary supervised cross-entropy epoch over its fixed local partition with batch size 128. The DQN [5] has fully connected layer widths $78 \to 256 \to 128 \to 4$ with ReLU hidden activations. Training uses Adam [6] with learning rate 10^{-3}, discount factor 0.99, replay capacity 50,000, initial exploration probability 1.0, multiplicative exploration decay 0.995, and minimum exploration probability 0.05. The encrypted paths use the same training schedule and real TenSEAL CKKS aggregation; only their aggregation transport differs from the corresponding plaintext paths.

The seed and partition policies differ between the two evaluations. Experiment 1 uses fixed seeds 42, 123, and 2025 while retaining the same pre-generated client partitions across seeds; its variability therefore reflects model initialization and stochastic training, conditional on those fixed partitions. Experiment 2 uses fixed seeds 41, 42, and 43. For each seed and client count, the same 9,000-row training pool is repartitioned using the per-class Dirichlet procedure [7] with concentration parameter 0.5; that seed also controls stochastic training. Its variability therefore includes both client-partition and training-trajectory variation.

3 Supplementary Tables

Table S1. Clean accuracy-deviation comparison with representative robust federated-learning defenses

Work	Main mechanism	Reported / computed clean deviation	Representative value
FLDetector [8]	Malicious-client detection and filtering	0–1.5 pp	0.23 pp avg.
FLAME [9]	HDBSCAN filtering, adaptive clipping, and noising	0.2–0.4 pp	0.30 pp avg.
Ours / TWA-PPA	**Trust-weighted aggregation + CKKS PPA**	**No loss;** 0.06 **pp mean paired absolute deviation vs.** ep_{twa}	0.06 **pp**
FLTrust [10]	Root-dataset trust bootstrapping	0–2 pp	0.83 pp avg.
FoolsGold [11]	Sybil-similarity learning-rate weighting	Not numerically tabulated; ≈ 0–2 pp	≈ 1.00 pp†

Note: "pp" denotes percentage points. Clean-accuracy deviation is measured relative to each work's closest matched clean/no-attack or plaintext baseline when available. For TWA-PPA, the matched baseline is plaintext TWA. Across seeds 42, 123, and 2025, the mean paired absolute round-30 difference is 0.000556, or 0.06 pp; the signed mean change is a 0.03 pp gain, so no loss is observed. † FoolsGold does not provide a clean numerical accuracy-loss table; the value is used only for visualization-level comparison.

Table S2. Clean global utility comparison

Execution Path	Accuracy	Macro-F1	Reward
ep_0	0.5472 ± 0.0588	0.5055 ± 0.0355	0.7361 ± 0.2942
ep_{sdp}	0.4977 ± 0.1164	0.4208 ± 0.1382	0.4883 ± 0.5820
ep_{twa}	$\mathbf{0.9674 \pm 0.0037}$	$\mathbf{0.9668 \pm 0.0038}$	$\mathbf{2.8372 \pm 0.0183}$
ep_{ppa}	0.5512 ± 0.0619	0.5124 ± 0.0445	0.7561 ± 0.3094
$ep_{\text{sdp-ppa}}$	0.5000 ± 0.1158	0.4227 ± 0.1375	0.5000 ± 0.5792
$ep_{\text{twa-ppa}}$	$\mathbf{0.9678 \pm 0.0033}$	$\mathbf{0.9671 \pm 0.0034}$	$\mathbf{2.8389 \pm 0.0164}$

Clean-utility note. Supplementary Table S2 reports the round-30 clean global utility of all six execution paths. Each path is evaluated using seeds 42, 123, and 2025 under the same five-benign-client configuration, and results are reported as mean ± standard deviation. Using multiple seeds reduces dependence on a particular model initialization or stochastic training trajectory and provides a measure of result variability. The evaluation reward follows the environment rule of +3 for a correct decision and −2 for an incorrect decision, yielding $R_{\text{eval}} = 5\,\text{Acc} - 2$. The trust-weighted paths provide the strongest clean utility: ep_{twa} achieves 0.9674 ± 0.0037 accuracy and 0.9668 ± 0.0038 macro-F1, while $ep_{\text{twa-ppa}}$ achieves 0.9678 ± 0.0033 accuracy and 0.9671 ± 0.0034 macro-F1.

Table S3.
Sensitivity of TWA to federation size and realized malicious-client ratio (mean ± SD over fixed seeds 41, 42, and 43)

Clients	Malicious ratio	Accuracy	Macro-F1	Mean mal. weight	Final mal. weight	Suppression	Conv. gain	Round time (s)
5	20%	0.893 ± 0.098	0.890 ± 0.102	0.0000 ± 0.0000	0.0000 ± 0.0000	1.000 ± 0.000	0.168 ± 0.085	2.687 ± 0.035
5	40%	0.591 ± 0.109	0.497 ± 0.122	0.2404 ± 0.0343	0.2401 ± 0.0210	0.044 ± 0.133	0.119 ± 0.118	2.690 ± 0.102
10	10%	0.868 ± 0.116	0.842 ± 0.155	0.0002 ± 0.0004	0.0005 ± 0.0010	0.867 ± 0.224	0.137 ± 0.181	5.231 ± 0.105
10	20%	0.859 ± 0.100	0.837 ± 0.136	0.0182 ± 0.0250	0.0384 ± 0.0757	0.378 ± 0.441	0.118 ± 0.166	5.180 ± 0.098
10	30%	0.855 ± 0.094	0.826 ± 0.133	0.0361 ± 0.0327	0.0632 ± 0.0726	0.178 ± 0.338	0.087 ± 0.055	5.387 ± 0.167
20	10%	0.915 ± 0.058	0.912 ± 0.062	0.0087 ± 0.0132	0.0121 ± 0.0177	0.378 ± 0.441	0.231 ± 0.077	10.648 ± 0.834
20	20%	0.883 ± 0.125	0.875 ± 0.142	0.0148 ± 0.0190	0.0282 ± 0.0480	0.156 ± 0.343	0.290 ± 0.141	10.153 ± 0.483
20	30%	0.769 ± 0.208	0.714 ± 0.283	0.0499 ± 0.0509	0.0965 ± 0.0989	0.000 ± 0.000	0.183 ± 0.240	10.005 ± 0.153

TWA sensitivity note. Supplementary Table S3 evaluates the sensitivity of TWA to federation size and the realized malicious-client ratio. Results are reported as mean ± standard deviation over the three fixed seeds. For $N = 5$, one and two malicious clients correspond to realized malicious-client ratios of 20% and 40%, respectively. Overall, increasing the malicious-client ratio tends to increase malicious aggregation influence and reduce the suppression rate. The largest degradation occurs for $N = 20$ with 30% malicious participation, where accuracy and macro-F1 decrease to 0.769 and 0.714, respectively, while the suppression rate falls to zero.

Table S4. Attack-path final reward and mean round time across fixed seeds 42, 123, and 2025

Panel A: Final Reward / Mean Round Time (s)—non-encrypted paths

Condition	ep_0	ep_{sdp}	ep_{twa}
∅	0.7361 ± 0.2942 / 2.8900 ± 0.0231	0.4883 ± 0.5820 / 2.9329 ± 0.0535	2.8372 ± 0.0183 / 2.9151 ± 0.0290
Sign flipping	1.5383 ± 0.8522 / 2.9273 ± 0.1149	1.0028 ± 0.2985 / 2.9404 ± 0.1211	2.8533 ± 0.0088 / 2.9572 ± 0.1765
RNB-Primary ($\alpha = 0.5$)	0.1844 ± 0.5894 / 3.0335 ± 0.1056	0.8233 ± 1.3037 / 3.0317 ± 0.0698	2.8539 ± 0.0038 / 3.0971 ± 0.1872
Model replacement	-0.9533 ± 0.3522 / 2.8870 ± 0.0129	1.2044 ± 0.2107 / 2.8723 ± 0.0216	2.8683 ± 0.0338 / 2.8666 ± 0.0140
Label flipping	0.9961 ± 0.5752 / 3.6809 ± 1.4664	0.3178 ± 0.4825 / 3.2438 ± 0.6807	2.8561 ± 0.0155 / 2.8859 ± 0.0488
Data poisoning	0.9767 ± 0.3355 / 2.8446 ± 0.0403	0.7489 ± 0.4781 / 2.8835 ± 0.0623	2.8506 ± 0.0042 / 2.8559 ± 0.0465
Reward poisoning	0.7361 ± 0.2942 / 2.8208 ± 0.0671	0.4883 ± 0.5820 / 2.8655 ± 0.0230	2.8406 ± 0.0136 / 2.8593 ± 0.0229

Panel B: Final Reward / Mean Round Time (s)—PPA-enabled paths

Condition	ep_{ppa}	$ep_{\text{sdp-ppa}}$	$ep_{\text{twa-ppa}}$
∅	0.7561 ± 0.3094 / 3.4150 ± 0.0161	0.5000 ± 0.5792 / 3.4223 ± 0.0172	2.8389 ± 0.0164 / 3.4948 ± 0.1315
Sign flipping	1.5322 ± 0.7965 / 3.5008 ± 0.1541	0.9761 ± 0.2917 / 3.4816 ± 0.1785	2.8522 ± 0.0086 / 3.4720 ± 0.1619
RNB-Primary ($\alpha = 0.5$)	0.1578 ± 0.5881 / 3.6612 ± 0.2577	0.8678 ± 1.3744 / 3.6186 ± 0.1734	2.8544 ± 0.0048 / 3.5459 ± 0.0626
Model replacement	-0.7494 ± 0.0010 / 3.4071 ± 0.0382	1.0861 ± 0.4215 / 3.4103 ± 0.0213	2.8567 ± 0.0213 / 3.3800 ± 0.0409
Label flipping	0.9628 ± 0.6796 / 3.4343 ± 0.0788	0.3200 ± 0.5287 / 3.4320 ± 0.0453	2.8567 ± 0.0173 / 3.4230 ± 0.0347
Data poisoning	1.1894 ± 0.5651 / 3.3992 ± 0.0315	0.7167 ± 0.4958 / 3.5281 ± 0.2453	2.8489 ± 0.0067 / 3.4204 ± 0.0387
Reward poisoning	0.7478 ± 0.3146 / 3.3532 ± 0.0642	0.4756 ± 0.5709 / 3.4149 ± 0.0634	2.8411 ± 0.0126 / 3.4291 ± 0.1163

Note: Values are mean ± standard deviation across seeds 42, 123, and 2025. Reward is taken from round 30.

4 Figures

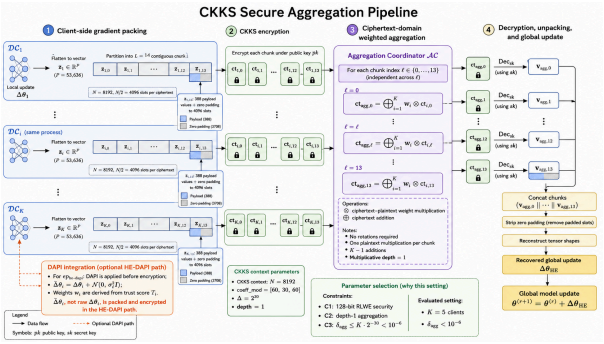

Figure S1. CKKS aggregation pipeline. Each update is split into $L = 14$ encrypted chunks and aggregated at $\mathcal{AC}$ with plaintext-scalar multiplication and ciphertext addition. $\mathcal{KA}$ decrypts only the aggregate chunks. Chunks 0–12 contain 4096 payload values; chunk 13 contains 388 values and 3708 zeros.

5 Reproducibility Note

Source code and scripts are available at:
`https://github.com/hetiantian10/TWAPPA-FRL`.

References

[1] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," in *Advances in Cryptology – ASIACRYPT 2017*, T. Takagi and T. Peyrin, Eds. Cham: Springer International Publishing, 2017, pp. 409–437.

[2] C. Peikert, "Lattice Cryptography for the Internet," in *Post-Quantum Cryptography*, M. Mosca, Ed. Cham: Springer International Publishing, 2014, pp. 197–219.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy*. SCITEPRESS, 2018, pp. 108–116. [Online]. Available: https://doi.org/10.5220/0006639801080116

[4] Canadian Institute for Cybersecurity, "CIC-IDS2017: Intrusion detection evaluation dataset," 2017. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html

[5] V. Mnih, K. Kavukcuoglu, D. Silver *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[6] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[7] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019. [Online]. Available: https://arxiv.org/abs/1909.06335

[8] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, "FLDetector: Defending Federated Learning Against Model Poisoning Attacks via Detecting Malicious Clients," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 2545–2555. [Online]. Available: https://doi.org/10.1145/3534678.3539231

[9] T. D. Nguyen, P. Rieger, H. Chen, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen, A. Mirhoseini, S. Zeitouni, F. Koushanfar, A.-R. Sadeghi, and T. Schneider, "FLAME: Taming Backdoors in Federated Learning," in *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 1415–1432. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/nguyen

[10] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2021.

[11] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "The Limitations of Federated Learning in Sybil Settings," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*. San Sebastian: USENIX Association, Oct. 2020, pp. 301–316. [Online]. Available: https://www.usenix.org/conference/raid2020/presentation/fung